Name:Srikar Putta ID number: 2520080038

Week 11

---

1. To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components

```
 GNU nano 8.7.1                          src/parser.c *
#include <stdio.h>
#include <string.h>
#include "parser.h"

int parse_command(char *input) {
    if (input == NULL || strlen(input) == 0) {
        printf("Parser: Empty command.\n");
        return 0;
    }

    printf("Parser: Command accepted -> %s\n", input);

    return 1;
}
```

```
 GNU nano 8.7.1                          src/executor.h *
#ifndef EXECUTOR_H
#define EXECUTOR_H

int execute_command(char *command);

#endif
```

```
 GNU nano 8.7.1                          src/parser.c *
#include <stdio.h>
#include <string.h>
#include "parser.h"

int parse_command(char *input) {
    if (input == NULL || strlen(input) == 0) {
        printf("Parser: Empty command.\n");
        return 0;
    }

    printf("Parser: Command accepted -> %s\n", input);

    return 1;
}
```

```
 GNU nano 8.7.1                          src/executor.c *
#include <stdio.h>
#include <stdlib.h>
#include "executor.h"

int execute_command(char *command) {
    int result;

    printf("Executor: Running command...\n");

    result = system(command);

    if (result != 0) {
        printf("Executor: Command failed.\n");
        return 0;
    }

    printf("Executor: Command completed successfully.\n");

    return 1;
}
```

```
 GNU nano 8.7.1                          src/main.c *
#include <stdio.h>
#include <string.h>

#include "parser.h"
#include "executor.h"

int main() {
    char command[100];

    printf("Enter command: ");
    fgets(command, sizeof(command), stdin);

    command[strcspn(command, "\n")] = '\0';

    if (!parse_command(command)) {
        printf("Main: Invalid command.\n");
        return 1;
    }

    if (!execute_command(command)) {
        printf("Main: Execution failed.\n");
        return 1;
    }

    printf("Main: End-to-end execution successful.\n");

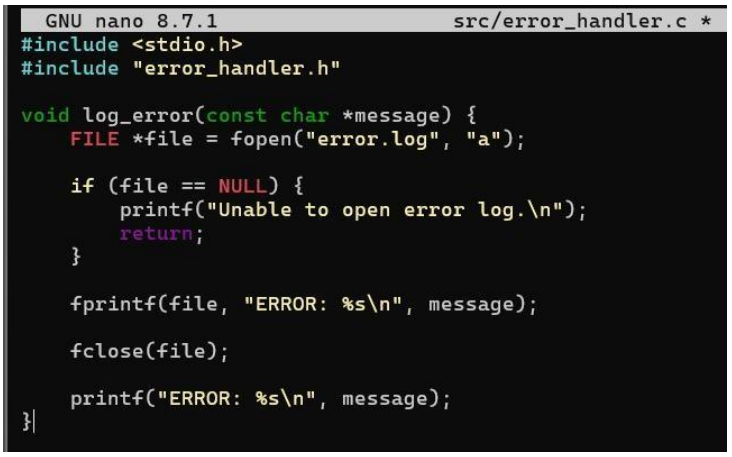
    return 0;
}
```

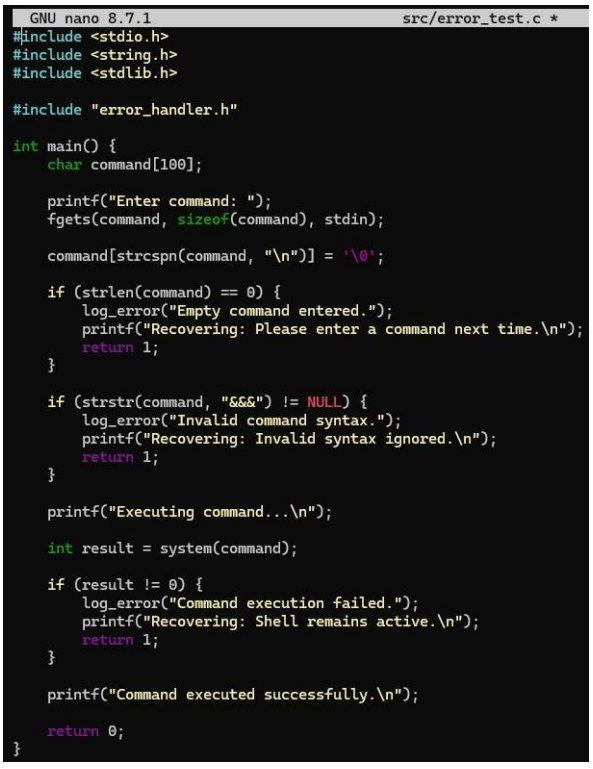
Output:

```
nocturnal@nocturnal:~$ ls
bin  src
nocturnal@nocturnal:~$ nano src/parser.h
nocturnal@nocturnal:~$ nano src/parser.c
nocturnal@nocturnal:~$ nano src/executor.h
nocturnal@nocturnal:~$ nano src/executor.c
nocturnal@nocturnal:~$ nano src/main.c
nocturnal@nocturnal:~$ gcc src/main.c src/parser.c src/executor.c -o bin/modular_shell
nocturnal@nocturnal:~$ ./bin/modular_shell
Enter command: ls
Parser: Command accepted -> ls
Executor: Running command...
bin  src
Executor: Command completed successfully.
Main: End-to-end execution successful.
nocturnal@nocturnal:~$ ./bin/modular_shell
Enter command: invalid_command_xyz
Parser: Command accepted -> invalid_command_xyz
Executor: Running command...
sh: 1: invalid_command_xyz: not found
Executor: Command failed.
Main: Execution failed.
```

2. To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios







Output:

```
/OS_Skill/Skill-Week-11$ nano src/error_handler.h
/OS_Skill/Skill-Week-11$ nano src/error_handler.c
/OS_Skill/Skill-Week-11$ nano src/error_test.c
```

```
Enter command: pwd
Executing command...
/home/sujit/OS_Skill/Skill-Week-11
Command executed successfully.
```

```
/OS_Skill/Skill-Week-11$ gcc src/error_test.c src/error_handler.c -o bin/e

/OS_Skill/Skill-Week-11$ ./bin/error_test
```

3. To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings

Output:

```
 GNU nano 8.7.1                                  src/memory_test.c *
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr;

    arr = malloc(5 * sizeof(int));

    if (arr == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < 5; i++)
        arr[i] = i + 1;

    printf("Array values: ");

    for (int i = 0; i < 5; i++)
        printf("%d ", arr[i]);

    printf("\n");

    free(arr);
    arr = NULL;

    printf("Memory released successfully.\n");

    return 0;
}
```

```
:~/OS_Skill/Skill-Week-11$ nano src/memory_test.c
:~/OS_Skill/Skill-Week-11$ gcc src/memory_test.c -o bin/memory_test -g
:~/OS_Skill/Skill-Week-11$ ./bin/memory_test
5
ssfully.
```

4. To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports

Output:

```
 GNU nano 8.7.1                                  src/test_memory.sh *
#!/bin/bash

echo "===== Memory Test Report ====="

echo "Test 1: Program execution"

output=$(./bin/memory_test)

echo "$output"

if echo "$output" | grep -q "Memory released successfully."; then
    echo "Test 1: PASS"
else
    echo "Test 1: FAIL"
fi

echo
echo "Test 2: Array values"

if echo "$output" | grep -q "1 2 3 4 5"; then
    echo "Test 2: PASS"
else
    echo "Test 2: FAIL"
fi

echo
echo "===== Test Summary ====="
echo "Testing completed."
```